ADR 001: Android SMS Receipt Capture

Oct 13, 2026 · @Doron Sinai

Status: Proposed. **This build (phase 1):** the Android app only. No backend, no accounts or login, no network calls. The app catches inbound SMS, decides which are receipts, and stores them on the device. Forwarding to the GCP ingest endpoint is designed here but deferred to a later phase. See PRD: Personal Grocery Price Intelligence for the full system.

Context

The phone is deliberately thin. It does not render or parse receipts and holds no price data. In this first build it does not even forward: it captures a receipt SMS, pulls out the link, and stores it on the device. Forwarding to GCP and everything downstream come in a later phase.

This has to be Android. iOS does not expose inbound SMS to third-party apps, so there is no capture path there. The app is sideloaded for personal use, which sidesteps the Play Store restriction on the SMS permission, since that restriction only applies to published apps.

Two constraints shape the design. First, inbound SMS includes highly sensitive traffic: one-time codes, bank alerts, two-factor messages. Whatever reads SMS can read all of it, so the app must keep (and later forward) as little as possible, and never act on a non-receipt. Second, Android restricts background work, so heavy work cannot happen inside the SMS receiver.

Decision Drivers

- Catch every inbound SMS reliably, including when the app is not open
- Forward only receipts, never OTPs, bank alerts or other personal SMS
- Do no network work in the receiver: hand off to a background job
- Survive process death, reboot and flaky connectivity without losing a receipt
- Forward the minimum: the link, not the message contents
- Authenticate to the backend so the ingest endpoint is not an open relay
- Be simple to sideload, run headless, and update merchant rules without a rebuild

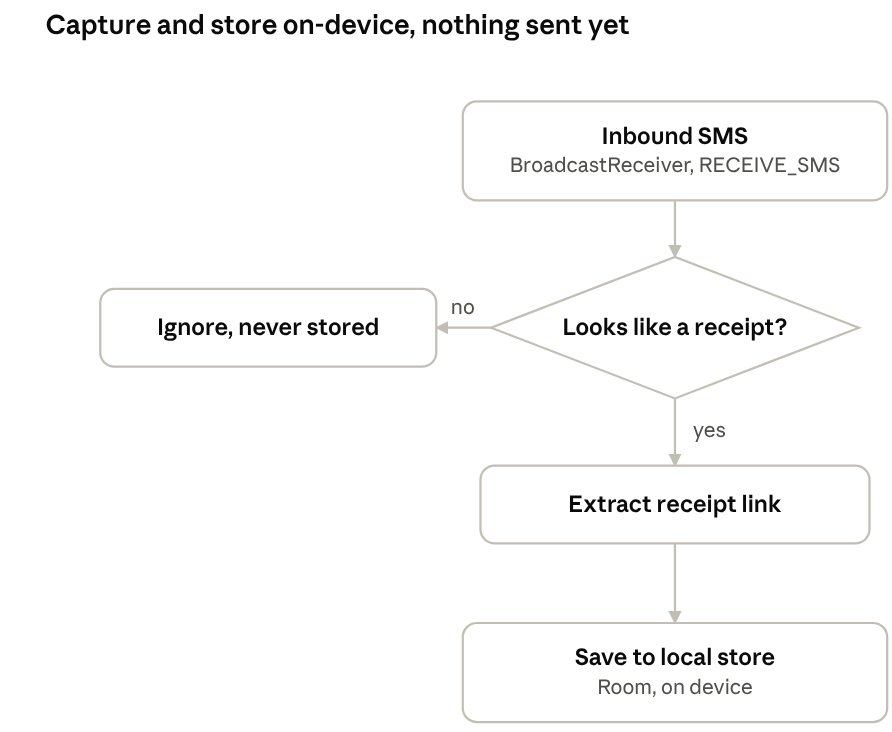
Decision

Build a small native Android app in Kotlin that captures receipts in three stages.

1. A `BroadcastReceiver` registered for `SMS_RECEIVED` (permission `RECEIVE_SMS`, not the default-SMS-app role) fires on every inbound message.
2. In the receiver, a cheap on-device filter decides whether the message looks like a receipt. Non-receipts are ignored at once. On a match, the receiver extracts the candidate link.
3. The receipt (link, sender, timestamp, a client id) is written to a local store on the device. No network, no account. A later phase adds a background job that forwards these stored records to the GCP ingest endpoint.

The authoritative allowlist (which domains are genuine receipts) will live server-side, as the PRD sets out. For this build there is no server, so the on-device filter is the only gate, and its job is to keep non-receipts out of the local store. Because nothing is forwarded yet, nothing leaves the device at all.

On-Device Capture Flow



on-device capture · filter, extract, forward

This first build stops at local storage. The filter is still the gate: non-receipts are dropped in the receiver and never stored. Forwarding to the backend is designed below but not built, so nothing leaves the phone.

Detection Rules

A message is a receipt candidate only if its body contains a link whose host is on the on-device merchant-domain list (for example shufersal.co.il, pairzon.com). That single rule is the pre-filter. The sender label (Shufersal, OSHERAD) is captured as a routing hint, not a gate, because alphanumeric sender IDs are spoofable.

Messages with no link, or with links only to unknown hosts, are ignored in the receiver and never forwarded. This is what keeps OTPs, bank alerts and personal SMS off the network entirely.

The on-device list is a conservative pre-filter, not the trust decision. The server still follows redirects and checks the final domain against the authoritative allowlist before anything is rendered.

Local Store, and the Deferred Forward

Stored record. For each receipt the app writes a small local record: the extracted link (or several, if the message carried more than one on-list link), the sender label, the message timestamp, and a stable client id. The raw message body is not stored.

No network in this build. Nothing is sent anywhere. There is no ingest call, no auth, no account. The record simply lands in a local store and stays on the device.

Shaped for the later forward. The record is deliberately the shape the GCP ingest endpoint will expect, so the deferred phase only has to add a background job (WorkManager, authenticated HTTPS, backoff, idempotency on the client id) that drains the store. Designing the schema now keeps that step cheap.

Alternatives Considered

Option	Verdict	Why
Native Android app	Chosen	Full control of the receiver, filter and retry; sideloadable; testable

Option	Verdict	Why
Tasker or MacroDroid (no-code)	Rejected	Fast to prototype, but awkward to version, test, and keep a tight forward-only filter
Become the default SMS app	Rejected	Grants far more than needed and changes the user's messaging; only required for the Play Store anyway
Network in the receiver	Rejected	onReceive has seconds before the process can be killed; loses receipts on flaky signal
Forward the full message body	Rejected	Larger egress of sensitive text; the link is all the server needs
Server-only filter (forward all SMS)	Rejected	Would send every OTP and bank alert off-device; unacceptable blast radius
iOS app	Rejected	iOS does not expose inbound SMS to third-party apps

Consequences

Good

- Nothing leaves the device at all in this build: receipts are captured and stored locally, nothing is sent
- Small and self-contained: no backend, no accounts, no network to stand up first
- The forward-only filter and link extraction are plain functions, easy to unit-test
- The local record is shaped for the later forward, so adding sync is a small, isolated change

Costs

- Receipts accumulate locally with no sync yet, so value is limited until the forward phase lands
- Not publishable to the Play Store as-is (SMS permission): sideload only, so updates are manual
- The on-device merchant list must be maintained as merchants and sender IDs change
- The app can read all SMS, so it is a sensitive artifact and must stay off shared devices

Scope Boundaries

This app does not:

- Render or screenshot the receipt page (later phase)
- Parse receipt contents or line items (later phase)
- Hold or query any price data (later phase)
- Scan barcodes or do the in-aisle lookup (a separate surface)
- Talk to any backend, or forward records off the device (deferred to the next phase)
- Have accounts, login or any user identity
- Make the authoritative allowlist decision (server-side, once the server exists)

Build Guidance for Claude Code

Language and tooling. Kotlin, Gradle (Kotlin DSL), single module. minSdk 26, targetSdk current. No Play Store config; local signing for sideload.

Components

- `SmsReceiver : BroadcastReceiver` on `SMS_RECEIVED`: reassemble multipart, run the filter, write the record. No network, no heavy work.
- `ReceiptFilter`: pure function, body to list of on-list links plus a sender routing hint. Fully unit-tested.
- `ReceiptStore`: local persistence (Room) of captured records, in the shape the later forward will use.
- `MerchantConfig`: the on-device sender hints and domain list, bundled with the app (a remote refresh can come later).
- `CapturedListScreen` (optional): a minimal screen to view captured receipts, for verifying the receiver works.
- `ForwardReceiptWorker` (deferred, not in this build): the future WorkManager job that drains `ReceiptStore` to the backend.

Manifest. `RECEIVE_SMS` only for this build (no `INTERNET` until forwarding lands); the receiver exported with `android:permission="android.permission.BROADCAST_SMS"`.

Libraries. Room for the local store, kotlinx-serialization for the record shape, Jetpack Compose only if the optional viewer screen is built. WorkManager and an HTTP client are added in the forward phase, not now.

Tests

- Unit: `ReceiptFilter` against the real example messages (Shufersal single link, OSHERAD two links including a bit.ly decoy, an OTP, a bank alert). Assert only on-list links come back.
- Unit: the stored record shape and dedupe id stability.
- Instrumented: feed a crafted `SMS_RECEIVED` intent to `SmsReceiver`; assert a record is written for a receipt and none for a non-receipt.

Open Decisions

- Local record shape: finalize the fields now so it maps one-to-one to the future ingest payload (link, sender, timestamp, client id).
- Build the optional captured-list screen for verification, or keep the app fully headless?
- Merchant config: a bundled list is fine for now; decide later whether a remote refresh is worth it.
- Multipart and non-Latin handling: confirm Hebrew and concatenated SMS reassemble correctly before link extraction.
- When a message has both an on-list and an off-list link (the Pairzon case): store only on-list links (the current rule), or store both for a later server-side decision?
- Minimum Android version to support, given the target device.
- How long to retain stored records before the forward phase exists (cap the local store, or keep everything)?